



Machine Learning Operations (MLOps)

E-commerce Product Recommendation System

Team no: 12

USN	Name
1RV23AI109	Suravi
1RV23AI110	Surya Pratap Singh
1RV23AI113	Tanuj S
1RV23AI115	Vaibhav S P



Agenda

- Introduction
- Motivation (Why this title)
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs



Introduction

Problem Statement

Rapid growth of e-commerce platforms has resulted in massive product catalogs

Recommendation systems play a critical role in:

- Personalizing user experience
- Increasing customer retention
- Driving business revenue

Challenges observed:

Users are often overwhelmed with choices, leading to:

- Poor product discovery
- Reduced engagement
- Lower conversion rates



Motivation

- Traditional recommendation models often has problems:
 - User searches for “*wireless headphones*”
 - Immediately after, recommendations still show **old interests** (e.g., shoes)
- Real-world systems require:
 - Low-latency predictions
 - Continuous performance monitoring
 - Reproducibility and scalability
- Motivation behind this project:
 - Bridge the gap between ML models and production systems
 - Apply end-to-end MLOps principles
 - Build a system suitable for large-scale e-commerce platforms



Problem Definition

- E-commerce platforms face the following challenges:
 - Information overload due to thousands of products
 - Generic recommendations with low relevance
 - Cold-start issues for new users and products
 - Difficulty in maintaining model performance over time
- Technical challenges include:
 - Handling sparse user–item interaction data
 - Real-time recommendation latency constraints
 - Monitoring data drift and model degradation

Hence, there is a need for a **scalable and monitored recommendation system** that operates reliably in production



Objectives

- **Build a personalized product recommendation system** that learns user preferences from historical interaction data to deliver relevant and user-specific product suggestions.
- **Implement multiple collaborative filtering models** (user-based, item-based, and matrix factorization) to capture diverse user–item interaction patterns.
- **Evaluate and compare models using ranking-based metrics** such as Precision@K, Recall@K, and NDCG@K to reflect real-world recommendation quality.
- **Integrate MLOps practices** including experiment tracking, data versioning, and automated pipelines to ensure traceability and efficient model lifecycle management.
- **Implement monitoring and drift detection mechanisms** to maintain model performance over time and ensure reproducibility, reliability, and deployment readiness.



Dataset Description

- **Dataset Used:** RetailRocket E-commerce Dataset
- **Dataset Characteristics:**
 - 2.7M+ user interaction events
 - 1.4M+ unique users
 - 235K+ unique items
- **Event Types:**
 - Product views
 - Add-to-cart events
 - Purchases
- **Key Challenge:**
 - Extremely sparse user–item interaction matrix (~99.98%)



Dataset Challenges

- Implicit feedback data (no explicit ratings)

- Highly imbalanced event distribution:

- Majority are product views

- Cold-start problem:

- New users with no history
- New products with few interactions

- Requires:

- Weighted interactions

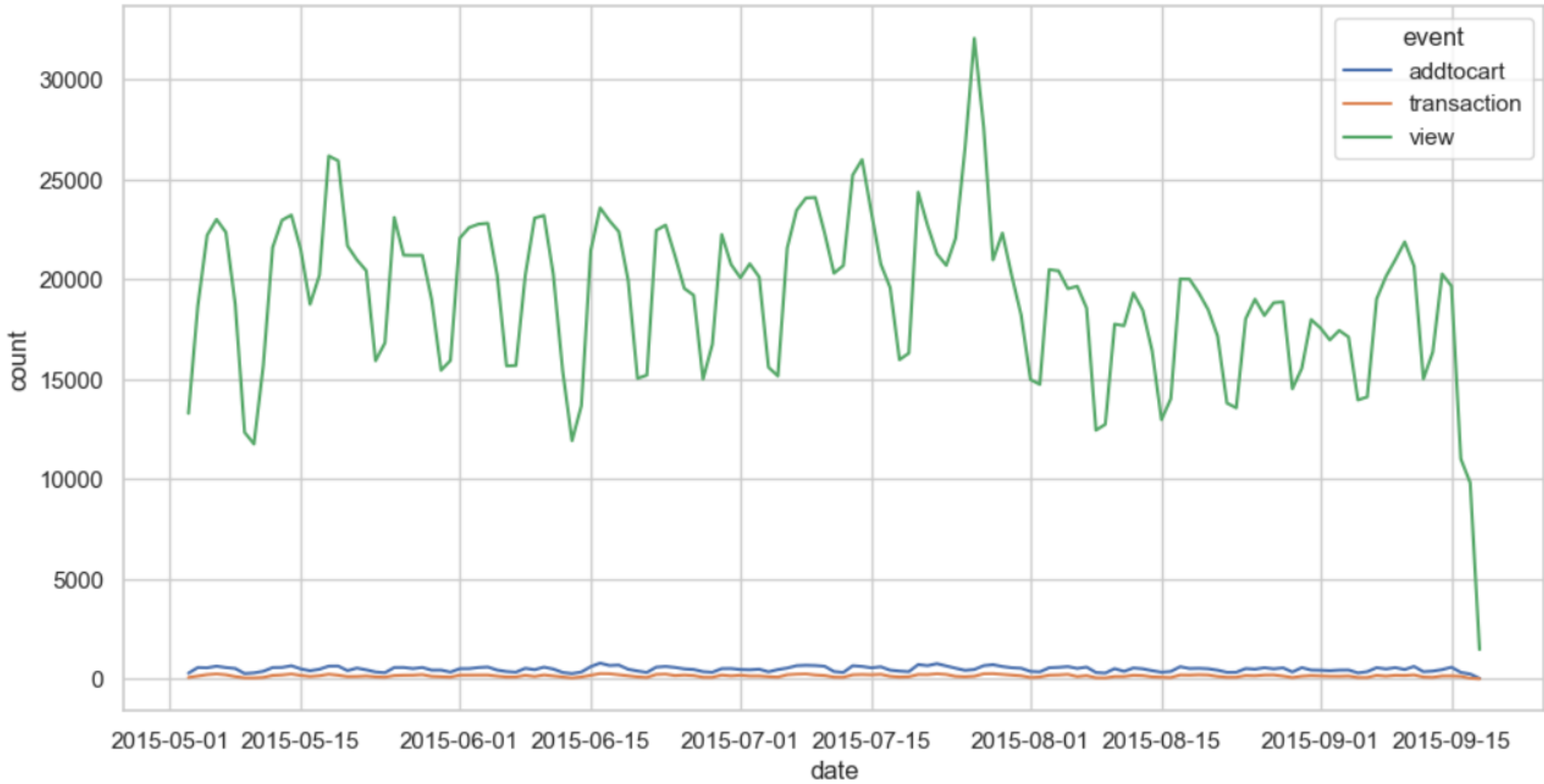
View:	1.0	
Add to Cart:	3.0	
Purchase:	5.0	

- Robust collaborative filtering techniques



EDA

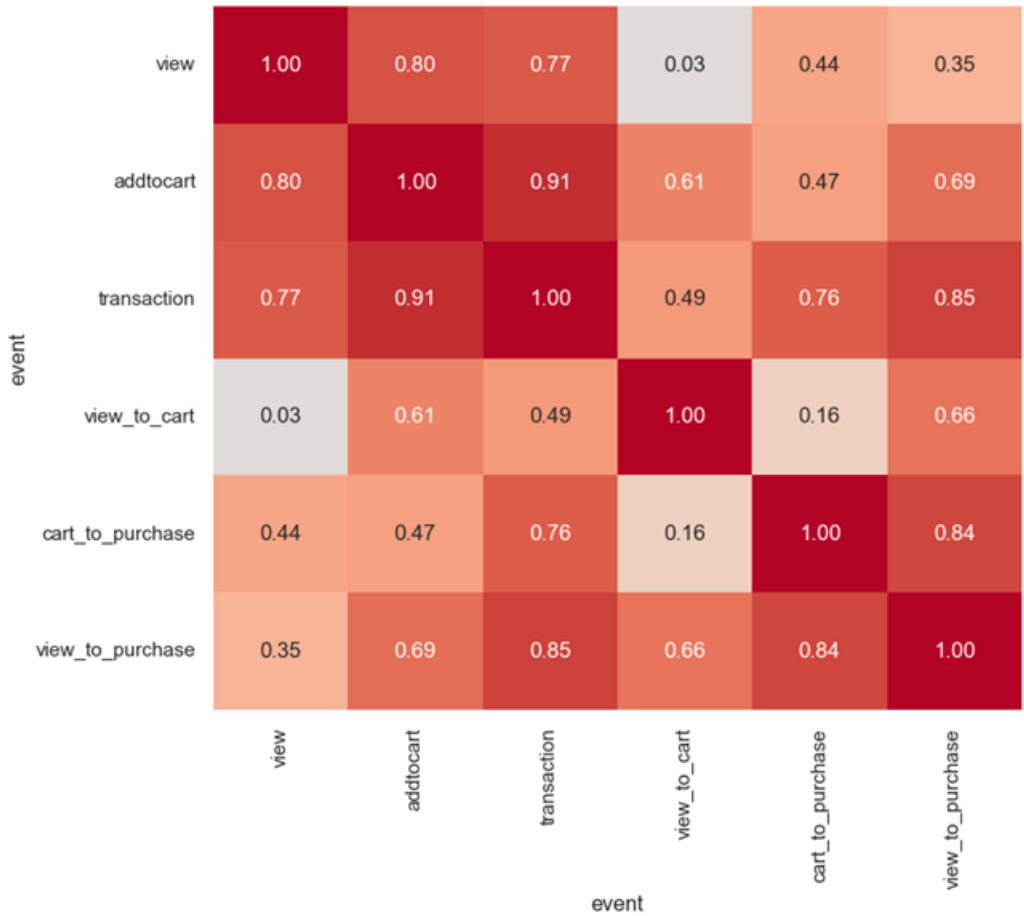
User activity by activity type over time



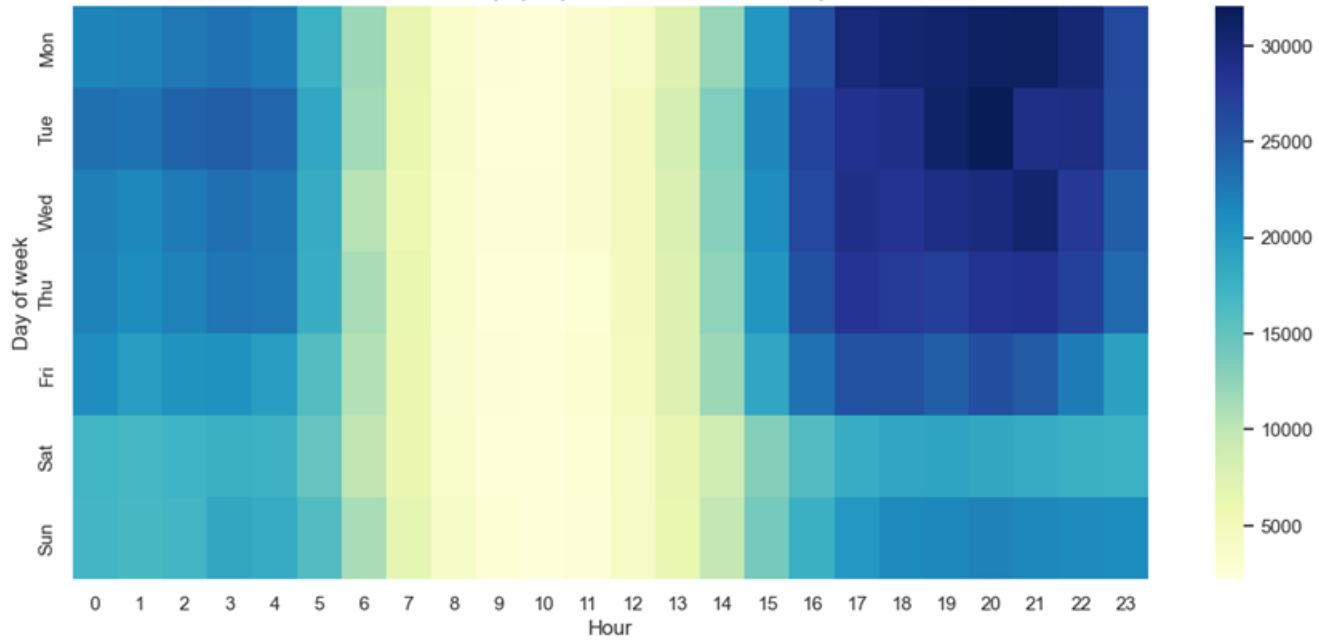


EDA

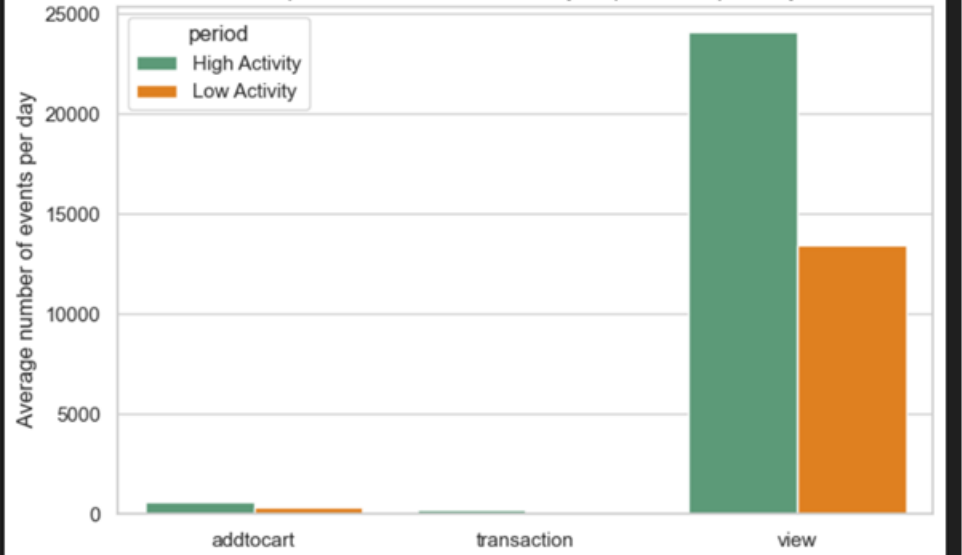
Correlation between events and conversion



User activity by day of the week and time of day



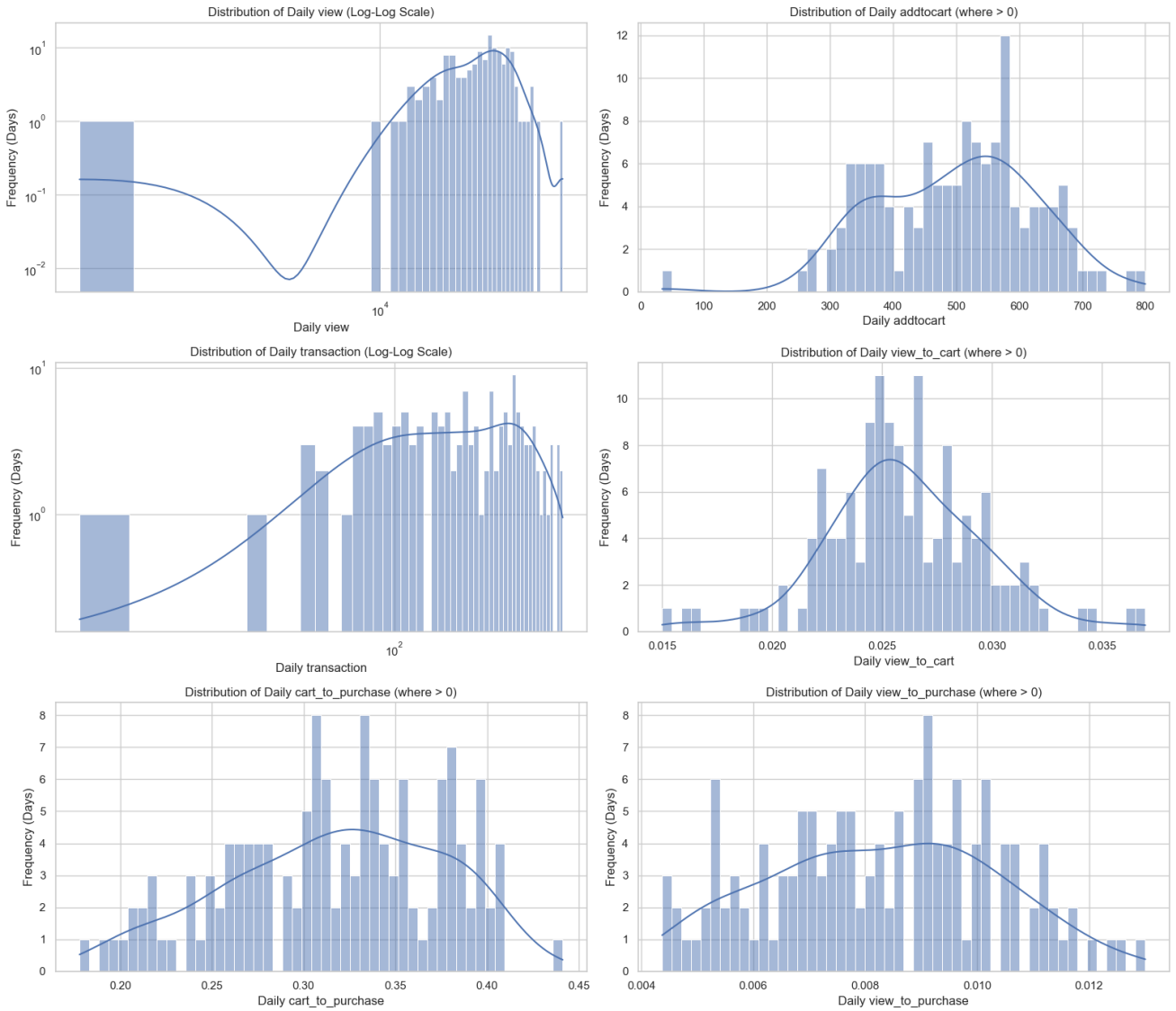
Comparison of interaction intensity on peak and quiet days





EDA

Distributions of Daily Activity & Conversion Rates





Risk Analysis

Identified Risks:

- Model performance degradation over time
- Data drift due to changing user behavior
- Loss of experiment reproducibility
- System integration failures
- Scalability limitations under high traffic

Mitigation Strategies:

- Continuous monitoring and drift detection
- Versioned datasets and models
- Automated experiment tracking
- Modular microservices architecture



Tools and Environment

Development Environment:

Programming Language: Python 3

Frontend: TypeScript

Version Control: Git

ML & Data Tools: Pandas, NumPy

Implicit Library (Collaborative Filtering)

Web Frameworks:

FastAPI (Backend & ML Service)

Next.js (Frontend)



MLOps and Monitoring Tools

- **DVC:** Dataset versioning and reproducibility
- **MLflow:** Experiment tracking and model registry
- **Docker:** Containerization
- **Prometheus:** Metrics collection
- **Grafana:** Visualization dashboards
- **Evidently AI:** Data and model drift detection
- **GitHub Actions:** CI/CD automation



Phase I - Machine Learning Models

- Focus on **collaborative filtering** techniques
- Models implemented:
 - Bayesian Personalized Ranking (BPR)
 - Alternating Least Squares (ALS)
 - Logistic Matrix Factorization (**LMF**)
- Designed for **implicit feedback data**
- Weighted interaction strategy:
 - View = 1
 - Add to Cart = 3
 - Purchase = 5



Evaluation Metrics

- **Precision@10:** 0.2838
 - Fraction of relevant items in top-10
- **Recall@10:** 0.3609
 - Fraction of relevant items retrieved
- **F1@10:** 0.2737
 - Harmonic mean of P&R
- **NDCG@10:** 0.4637
 - Position-aware ranking quality
- **MAP:** 0.3288
 - Mean Average Precision
- **MRR:** 0.7681
 - Mean Reciprocal Rank
- **Hit Rate@10:** 0.9452
 - % users with ≥ 1 relevant item



Model Comparison

- **BPR (Bayesian Personalized Ranking)**
 - Best overall ranking performance
 - Specifically optimized for implicit feedback
 - Strong results on NDCG, MAP, and MRR
- **ALS (Alternating Least Squares)**
 - Highly scalable and computationally efficient
 - Suitable for large datasets
 - Slightly lower ranking quality compared to BPR
- **LMF (Logistic Matrix Factorization)**
 - Provides a balanced trade-off between precision and recall
 - More stable than ALS, but not top-performing

Final Selection

✓ BPR chosen as the production model



Phase-II Building ML Ops Pipeline

- **End-to-End ML Lifecycle Implemented**

- **Data Ingestion**

Automated pipelines ingest raw user–item interaction data from source systems into a centralized storage layer.

- **Feature Engineering**

Data is cleaned, transformed, and converted into model-ready features with consistent preprocessing logic.

- **Model Training**

Multiple collaborative filtering models are trained using reproducible configurations and controlled experiments.

- **Model Evaluation**

Models are evaluated using ranking-based metrics to ensure real-world recommendation quality.

- **Versioning**

Datasets, features, and models are versioned to track changes and enable rollback when required.

- **Monitoring**

Continuous monitoring detects data drift, performance degradation in production.



Phase-II Building ML Ops Pipeline

	Component	MLOPs Tools
1	Data Management	DVC, PostgreSQL
2	Experiment Tracking and Version Control	WanDB, Github
3	Automation and Pipeline Orchestration	FastAPI, Github
4	Model Deployment & Serving	FastAPI, Docker
5	Monitoring & Governance and collaboration	Prometheus, Grafana, Evidently AI



System Architecture

- Frontend**

Next.js web application for responsive UI and real-time user interactions

- Backend**

FastAPI service handling business logic, user requests, and orchestration

- ML Service**

Dedicated FastAPI-based recommendation service for real-time inference

- Database**

PostgreSQL for storing user data, item metadata, and interaction logs

- Model Registry**

MLflow for model versioning, experiment tracking, and deployment management

- Monitoring Stack**

Prometheus for metrics collection and Grafana for visualization and alerts



Experiment Tracking

- MLflow Usage in the Project
- **MLflow is used to log:**
 - **Hyperparameters** – Model configuration and training settings
 - **Metrics** – Evaluation results for performance comparison
 - **Model Artifacts** – Trained models and related files
- This enables:
 - **Reproducibility** – Experiments can be reliably re-run
 - **Model Comparison** – Easy benchmarking across multiple models
 - **Model Version Control** – Track, register, and manage model versions
- **Git is used for:**
 - **Source Code Versioning** – Tracks changes in data pipelines, feature engineering, and model training code
 - **Collaboration** – Enables team-based development with branching and pull requests
 - **Change Traceability** – Maintains a complete history of code evolution



Monitoring and Drift Detection

- **DVC**
 - Version control for datasets and features
 - Ensures data reproducibility and experiment traceability
- **Prometheus**
 - Tracks API latency, request counts, and error rates
 - Provides real-time metrics for system performance
- **Grafana**
 - Visual dashboards for system health monitoring
 - Enables alerting and performance analysis
- **Evidently AI**
 - Detects data drift and model performance degradation
 - Ensures long-term reliability of the ML system



Business Impact

- **Improved Product Discovery**

Personalized recommendations help users find relevant products faster

- **Increased User Engagement**

Users interact more frequently due to relevant and timely suggestions

- **Higher Conversion Rates**

Targeted recommendations increase the likelihood of purchases

- **Revenue Uplift Through Personalization**

Better matching of users to products drives incremental revenue

- **Scalable for Real-World Deployment**

Production-ready architecture supports growth in users and data volume

- **Scalable & Production-Ready**

Designed to handle real-world traffic and data growth

- **Revenue Growth via Personalization**

Data-driven recommendations contribute to incremental revenue uplift



Conclusion

- Designed and implemented a scalable, end-to-end e-commerce recommendation system
- Applied core ML concepts including collaborative filtering, ranking-based evaluation, and model selection
- Implemented production-grade MLOps practices:
 - Automated training and evaluation pipelines
 - Data, code, and model versioning
 - Continuous monitoring and drift detection
- Ensured the system is:
 - Reproducible across experiments and deployments
 - Observable through metrics and dashboards
 - Extensible for future models and features
- Demonstrates how MLOps bridges the gap between research models and real-world ML systems